

《系统开发工具基础》实验报告



中国海洋大学
OCEAN UNIVERSITY OF CHINA

学号： 24120011027

姓名： 严程漪

专业： 物流管理

我的 github: [点击这里](#)

2026 年 9 月 21 日

目录

1	实验目的	2
1.1	理解学习代码质量	2
1.2	理解学习元编程	2
1.3	理解学习大杂烩	2
2	课内实验	2
2.1	第一题：建立可执行的本地质量门禁	2
2.2	第二题：让 Make 只重建真正受影响的产物	4
2.3	第三题：把本地 API 数据转换为可读报告	5
2.4	第四题：修复并交付一个陌生的小型工具仓库	7
3	课外实例	9
3.1	实例一：配置格式化器、linter 和 pre-commit 钩子	9
3.2	实例二：单元测试 + 代码覆盖率	12
3.3	实例三：持续集成	15
3.4	实例四：grep 正则找危险代码	17
3.5	实例五：Markdown 项目符号替换	18
3.6	实例六：正则提取 JSON 的 name	19
3.7	实例七：Makefile clean 目标	20
3.8	实例八：Rust 版本要求语法	21
3.9	实例九：pre-commit 钩子跑 make paper.pdf	21
3.10	实例十：shellcheck+GitHub Action	24
4	解题感悟	25
5	commit 记录	26

1 实验目的

1.1 理解学习代码质量

1.2 理解学习元编程

1.3 理解学习大杂烩

2 课内实验

2.1 第一题：建立可执行的本地质量门禁

1、复制 q10 为 q13; 在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。

```
ouc@islouc-vm:~/q13$ cat << 'EOF' > pyproject.toml
> [build-system]
> requires = ["setuptools>=68"]
> build-backend = "setuptools.build_meta"
>
> [project]
> name = "greetlab-24120011027"
> version = "0.1.0"
>
> [project.scripts]
> sdt-greet = "greetlab.cli:main"
>
> [tool.ruff]
> line-length = 88
>
> [tool.pytest.ini_options]
> testpaths = ["tests"]
> pythonpath = ["src"]
> EOF
```

图 1: pyproject.toml

2、在 cli.py 中补充空白姓名检验, 每个测试包含有效断言。

```
ouc@islouc-vm:~/q13$ cat << 'EOF' > tests/test_cli.py
> import subprocess
> import sys
> def test_normal_name():
>     result = subprocess.run(
>         [sys.executable, "-m", "greetlab.cli", "--name", "Alice"],
>         capture_output=True,
>         text=True,
>         check=False,
>     )
>     assert result.returncode == 0
>     assert "Hello, Alice" in result.stdout
> def test_blank_name_raise_exit2():
>     result = subprocess.run(
>         [sys.executable, "-m", "greetlab.cli", "--name", ""],
>         capture_output=True,
>         text=True,
>         check=False,
>     )
>     assert result.returncode == 2
> EOF
```

图 2: test_cli.py

3、运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则。运行 ruff format 最开始得到的答案是：

```
ouc@islouc-vm:~/q13$ ruff format .
error: Failed to parse src/greetlab/cli.py:10:5: Invalid annotated assignment target
3 files reformatted, 4 files left unchanged
```

图 3: 报错截图

可以看出出错位置在 cli.py 文件的第 10 行，检查代码发现是输入时将”输成了”，导致的错误。调整之后，再次运行，发现均正常进行

```
ouc@islouc-vm:~/q13$ ruff format .
4 files left unchanged
ouc@islouc-vm:~/q13$ ruff check
All checks passed!
ouc@islouc-vm:~/q13$ python3 -m pytest
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q13
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.05 seconds =====
```

图 4: 正确运行截图

4、编写 check.sh，依次执行 ruff format --check 和 pytest；运行并保证返回 0。首先是编写 check.sh，并赋予执行权限。

```
ouc@islouc-vm:~/q13$ cat << 'EOF' > check.sh
> #!/usr/bin/env bash
> set -euo pipefail
> ruff format --check .
> ruff check .
> pytest
> EOF
ouc@islouc-vm:~/q13$ chmod +x check.sh
```

图 5: check.sh

依次执行 ruff format --check 和 pytest，运行并记录结果。

```
ouc@islouc-vm:~/q13$ ruff format --check
4 files already formatted
ouc@islouc-vm:~/q13$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q13
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.05 seconds =====
ouc@islouc-vm:~/q13$ echo $?
0
```

图 6: 结果截图

返回确实是 0。

2.2 第二题：让 Make 只重建真正受影响的产物

1、创建给定的 data.csv、stats.py、build_report.py 和 report.md。

```
ouc@islouc-vm:~$ mkdir -p q14 && cd q14
ouc@islouc-vm:~/q14$ cat << 'EOF' > data.csv
> name,value
> alpha,2
> beta,3
> gamma,5
> EOF
ouc@islouc-vm:~/q14$ cat << 'EOF' > stats.py
> import csv
> with open("data.csv") as f:
>     total = sum(int(r["value"]) for r in csv.DictReader(f))
> open("stats.txt", "w").write(str(total))
> EOF
ouc@islouc-vm:~/q14$ cat << 'EOF' > build_report.py
> text = open("report.md").read()
> total = open("stats.txt").read()
> open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
> EOF
ouc@islouc-vm:~/q14$ cat << 'EOF' > report.md
> # Data Report
> EOF
```

图 7: 文件创建

2、编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。

```
ouc@islouc-vm:~/q14$ cat << 'EOF' > Makefile
> .PHONY: all clean
> all: stats.txt report.txt
> stats.txt: data.csv stats.py
>     python3 stats.py
> report.txt: report.md stats.txt build_report.py
>     python3 build_report.py
> clean:
>     rm -f stats.txt report.txt
> EOF
```

图 8: Makefile 文件

然后运行：cat -A Makefile 和 sed -i 's/^[[:space]]\+/\t/'，再运行 cat -A，让 Makefile 的缩进改好，因为 python 是用缩进区分块的，这样让 make 能正常运行。

```
.PHONY: all clean$
all: stats.txt report.txt$
stats.txt: data.csv stats.py$
^Ipython3 stats.py$
report.txt: report.md stats.txt build_report.py$
^Ipython3 build_report.py$
clean:$
^Irm -f stats.txt report.txt$
```

图 9: 运行结果

3、验证首次 make 生成两个产物；第二次无改动时不执行配方。

```
ouc@islouc-vm:~/q14$ make
python3 stats.py
python3 build_report.py
ouc@islouc-vm:~/q14$ make
make: Nothing to be done for 'all'.
```

图 10: 结果截图

4、touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

touch 把 data.csv 的修改时间改成“现在”，将两个产物都重建，再运行 make

```
ouc@islouc-vm:~/q14$ touch data.csv
ouc@islouc-vm:~/q14$ make
python3 stats.py
python3 build_report.py
ouc@islouc-vm:~/q14$ cat stats.txt
10ouc@islouc-vm:~/q14$ cat report.txt
# Data Report

Total: 10
```

图 11: 结果截图

使用 make clean 只删除生成物，用 ls 检查是否正确。

```
ouc@islouc-vm:~/q14$ make clean
rm -f stats.txt report.txt
ouc@islouc-vm:~/q14$ ls
build_report.py data.csv Makefile report.md stats.py
```

图 12: 结果截图

2.3 第三题：把本地 API 数据转换为可读报告

1、把给定 JSON 保存 packages.json，并在 q15 目录运行 python -m http.server 8000。

```
ouc@islouc-vm:~/q15$ cat << 'EOF' > packages.json
> [
>   {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
>   {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
>   {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
>   {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
>   {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
> ]
> EOF
```

图 13: packages.json

原本应该运行 8000，但是后面运行时，会发现一直显示占用，所以后面我使用到了 8001。

```
ouc@islouc-vm:~/q15$ python3 -m http.server 8001
Serving HTTP on 0.0.0.0 port 8001 (http://0.0.0.0:8001/) ...
127.0.0.1 - - [14/Sep/2026 03:12:41] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 03:12:59] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 03:14:06] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 03:15:21] "GET /packages.json HTTP/1.1" 200 -
```

图 14: 运行截图

2、使用 curl -fsS 获取 http://127.0.0.1:8001/packages.json。

```
ouc@islouc-vm:~/q15$ curl -fsS http://127.0.0.1:8001/packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

图 15: 结果截图

3、使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录，并按 downloads 降序、name 升序排序。

```
ouc@islouc-vm:~/q15$ curl -fsS http://127.0.0.1:8001/packages.json \
> | jq '[.[] | select(.status == "active" and .downloads >= 100)] | sort_by(-.downloads, .name)'
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]
```

图 16: 筛选结果

4、编写 api_report.sh, 把结果生成 summary.md, 包含标题和 name/version/downloads 三列表格。先编写 api_report.sh, 并赋予执行权限。

```
ouc@islouc-vm:~/q15$ cat << 'EOF' > api_report.sh
> #!/usr/bin/env bash
> set -euo pipefail
>
> {
>   echo "# Package Report"
>   echo
>   echo "| name | version | downloads |"
>   echo "|-----|-----|-----|"
>   curl -fsS http://127.0.0.1:8001/packages.json \
>   | jq -r '[.[] | select(.status == "active" and .downloads >= 100)] | sort_by(-.downloads, .name)'
> } | "| \(.name) | \(.version) | \(.downloads) |"
> } > summary.md
> EOF
ouc@islouc-vm:~/q15$ chmod +x api_report.sh
```

图 17: api_report.sh

生成 summary.md, 并记录表格

```
ouc@islouc-vm:~/q15$ ./apt_report.sh
ouc@islouc-vm:~/q15$ cat summary.md
# Package Report

| name | version | downloads |
|-----|-----|-----|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 18: 表格

2.4 第四题：修复并交付一个陌生的小型工具仓库

- 1、复制 q13 为 q16 并初始化/继续使用 Git

```
ouc@islouc-vm:~/q15$ cd ..
ouc@islouc-vm:~$ cp -r q13 q16
ouc@islouc-vm:~$ cd q16
ouc@islouc-vm:~/q16$ git init
Initialized empty Git repository in /home/ouc/q16/.git/
```

图 19: 运行截图

- 2、把问候语实现临时改成始终返回字面量 Hello,name!, 运行 check.sh 确认测试失败; 随后定位并修复。

```
ouc@islouc-vm:~/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q16
collected 2 items

tests/test_cli.py F. [100%]

===== FAILURES =====
_____ test_normal_name _____

    def test_normal_name():
        result = subprocess.run(
            [sys.executable, "-m", "greetlab.cli", "--name", "Alice"],
            capture_output=True,
            text=True,
            check=False,
        )
        assert result.returncode == 0
>       assert "Hello, Alice" in result.stdout
E       AssertionError: assert 'Hello, Alice' in 'Hello, name!\n'
E       + where 'Hello, name!\n' = CompletedProcess(args=['/usr/bin/python3', '-m', 'greetlab.cli', '--name', 'Alice'], returncode=0, stdout='Hello, name!\n', stderr='').stdout

tests/test_cli.py:13: AssertionError
===== 1 failed, 1 passed in 0.06 seconds =====
```

图 20: 错误截图

将实现改为 `f"Hello,a.name"`，再次运行 `./check.sh`，确认测试全部恢复通过

```
ouc@islouc-vm:~/q16$ ruff format .
1 file reformatted, 3 files left unchanged
ouc@islouc-vm:~/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q16
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.05 seconds =====
ouc@islouc-vm:~/q16$ echo $?
0
```

图 21: 通过截图

3、编写最小 Makefile，包含 check、build 和 clean；check 调用 check.sh，build 生成 wheel。

```
Visual Studio Code
ouc@islouc-vm:~/q16$ cat << 'EOF' > Makefile
> .PHONY: check build clean
> check:
>     ./check.sh
> build:
>     python3 -m build
> clean:
>     rm -rf dist build src/*.egg-info
> EOF
ouc@islouc-vm:~/q16$ sed -i 's/^[[:space:]]\+/\t/' Makefile
ouc@islouc-vm:~/q16$ cat -A Makefile
.PHONY: check build clean$
check:$
^I./check.sh$
build:$
^Ipython3 -m build$
clean:$
^Irm -rf dist build src/*.egg-info$
```

图 22: 最小 Makefile

4、运行 make check 和 make build，计算 wheel 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。

```
ouc@islouc-vm:~/q16$ sha256sum dist/*.whl
2834523d394eaa338a61084b7105cf34951e5204349b4765a4fbb3779d5db4f3 dist/greetlab_24
120011027-0.1.0-py3-none-any.whl
```

图 23: SHA-256

编写.gitignore 排除构建产物与缓存，执行 git add -A 后，以内容聚焦的提交信息完成提交，并用 git log -oneline 验证提交记录。

```
ouc@islouc-vm:~/q16$ cat << 'EOF' > .gitignore
> __pycache__/
> *.pyc
> dist/
> build/
> *.egg-info/
> .pytest_cache/
> EOF
ouc@islouc-vm:~/q16$ git add -A
```

图 24: .gitignore

执行了 `git add -A` 后, 再执行 `git status` 和 `git commit -m "Add quality gate and Makefile, fix blank name handling"` 最后使用 `git log --oneline` 验证提交记录

```
ouc@islouc-vm:~/q16$ git log --oneline
ed200f7 (HEAD -> master) Add quality gate and Makefile, fix blank name handling
ouc@islouc-vm:~/q16$
```

图 25: 提交记录

3 课外实例

3.1 实例一：配置格式化器、linter 和 pre-commit 钩子

1、先编写一个故意带错误的代码。

```
ouc@islouc-vm:~/hw1$ cat > mypackage/calculator.py << 'EOF'
> """一个用于演示的简单计算器模块。"""
>
>
> def add(a, b):
>     result=a+b
>     return result
>
>
> def divide(a, b):
>     if b==0:
>         raise ValueError('cannot divide by zero')
>     return a/b
>
>
> def shout(message):
>     x = 42
>     return message.upper()+ '!!!'
>
>
> def append_item(item, lst=[]):
>     lst.append(item)
>     return lst
> EOF
```

图 26: calculator.py

2、运行 linter, 观察报错。

```
ouc@islouc-vm:~/hw1$ python3 -m ruff check mypackage/
F841 Local variable 'x' is assigned to but never used
--> mypackage/calculator.py:16:5
15 | def shout(message):
16 |     x = 42
   |     ^
17 |     return message.upper()+ '!!!'
help: Remove assignment to unused variable 'x'

B006 Do not use mutable data structures for argument defaults
--> mypackage/calculator.py:20:27
20 | def append_item(item, lst=[]):
   |                        ^^
21 |     lst.append(item)
22 |     return lst
help: Replace with 'None'; initialize within function

Found 2 errors.
No fixes available (2 hidden fixes can be enabled with the '--unsafe-fixes' option).
```

图 27: 报错截图

3、运行 formatter 自动修复。

```
ouc@islouc-vm:~/hw1$ python3 -m ruff format mypackage/calculator.py
1 file reformatted
```

图 28: 结果截图

4、用 AI 帮助修复所有 linter 错误。

```
ouc@islouc-vm:~/hw1$ cat > mypackage/calculator.py << 'EOF'
> """一个用于演示的简单计算器模块。"""
>
>
> def add(a, b):
>     result = a + b
>     return result
>
>
> def divide(a, b):
>     if b == 0:
>         raise ValueError("cannot divide by zero")
>     return a / b
>
>
> def shout(message):
>     return message.upper() + "!!!"
>
>
> def append_item(item, lst=None):
>     if lst is None:
>         lst = []
>         lst.append(item)
>     return lst
> EOF
ouc@islouc-vm:~/hw1$ python3 -m ruff check mypackage/
All checks passed!
```

图 29: 结果截图

5、写 pre-commit 钩子。

```
ouc@islouc-vm:~/hw1$ cat > .git/hooks/pre-commit << 'EOF'
> #!/bin/bash
> echo "[pre-commit] 正在运行代码检查..."
>
> python3 -m ruff format --check .
> if [ $? -ne 0 ]; then
>     echo "[pre-commit] ❌ 格式不规范, 请先跑: python3 -m ruff format ."
>     exit 1
> fi
>
> python3 -m ruff check .
> if [ $? -ne 0 ]; then
>     echo "[pre-commit] ❌ linter 发现错误, 请修复后再提交"
>     exit 1
> fi
>
> echo "[pre-commit] ✅ 所有检查通过, 允许提交"
> exit 0
> EOF
ouc@islouc-vm:~/hw1$
ouc@islouc-vm:~/hw1$ chmod +x .git/hooks/pre-commit
```

图 30: pre-commit

6、第一次正常提交。

```
ouc@islouc-vm:~/hw1$ chmod +x .git/hooks/pre-commit
ouc@islouc-vm:~/hw1$ python3 -m ruff format .
2 files left unchanged
ouc@islouc-vm:~/hw1$ git add -A
ouc@islouc-vm:~/hw1$ git commit -m "first commit"
[pre-commit] 正在运行代码检查...
2 files already formatted
All checks passed!
[pre-commit] ✓ 所有检查通过, 允许提交
On branch master
nothing to commit, working tree clean
```

图 31: 结果截图

7、故意埋雷，验证钩子拦截。

```
ouc@islouc-vm:~/hw1$ cat >> mypackage/calculator.py << 'EOF'
>
> def evil():
>     unused = 999
>     return 1
> EOF
ouc@islouc-vm:~/hw1$
ouc@islouc-vm:~/hw1$ git add -A
ouc@islouc-vm:~/hw1$ git commit -m "add evil"
[pre-commit] 正在运行代码检查...
2 files already formatted
F841 Local variable `unused` is assigned to but never used
--> mypackage/calculator.py:27:5
26 | def evil():
27 |     unused = 999
    |     ^^^^^
28 |     return 1
    |
help: Remove assignment to unused variable `unused`

Found 1 error.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).
[pre-commit] ✗ linter 发现错误, 请修复后再提交
```

图 32: 结果截图

8、修复。

```
ouc@islouc-vm:~/hw1$ cat > mypackage/calculator.py << 'EOF'
> """一个用于演示的简单计算器模块。"""
>
> def add(a, b):
>     result = a + b
>     return result
>
> def divide(a, b):
>     if b == 0:
>         raise ValueError("cannot divide by zero")
>     return a / b
>
> def shout(message):
>     return message.upper() + "!!!"
>
> def append_item(item, lst=None):
>     if lst is None:
>         lst = []
>     lst.append(item)
>     return lst
> EOF
ouc@islouc-vm:~/hw1$
ouc@islouc-vm:~/hw1$ python3 -m ruff format .
2 files left unchanged
ouc@islouc-vm:~/hw1$ git add -A
ouc@islouc-vm:~/hw1$ git commit -m "clean up"
[pre-commit] 正在运行代码检查...
2 files already formatted
All checks passed!
[pre-commit] 所有检查通过，允许提交
On branch master
nothing to commit, working tree clean
```

图 33: 结果截图

9、最终验证。

```
ouc@islouc-vm:~/hw1$ git status
On branch master
nothing to commit, working tree clean
```

图 34: 结果截图

3.2 实例二：单元测试 + 代码覆盖率

1、写一个不全的测试。

```
ouc@islouc-vm:~/hw1$ cat > tests/test_calculator.py << 'EOF'
> from mypackage import add, divide
>
> def test_add():
>     assert add(2, 3) == 5
>     assert add(-1, 1) == 0
>
> def test_divide_normal():
>     assert divide(10, 2) == 5
> EOF
```

图 35: test_calculator.py

2、运行，看初始覆盖率。

```
ouc@islouc-vm:~/hw1$ python3 -m pytest tests/ -v --cov=mypackage --cov-report=term
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0 -- /usr/bin
/python3
cachedir: .pytest_cache
rootdir: /home/ouc/hw1
plugins: cov-5.0.0
collected 2 items

tests/test_calculator.py::test_add PASSED [ 50%]
tests/test_calculator.py::test_divide_normal PASSED [100%]

----- coverage: platform linux, python 3.8.10-final-0 -----
Name                               Stmts  Miss  Cover
-----
mypackage/__init__.py                1     0   100%
mypackage/calculator.py             14     6    57%
-----
TOTAL                               15     6    60%

===== 2 passed in 0.02 seconds =====
```

图 36: 初始覆盖率

3、查看具体漏了那些行。

```
ouc@islouc-vm:~/hw1$ python3 -m pytest tests/ --cov=mypackage --cov-report=term-mi
ssing
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/hw1
plugins: cov-5.0.0
collected 2 items

tests/test_calculator.py .. [100%]

----- coverage: platform linux, python 3.8.10-final-0 -----
Name                               Stmts  Miss  Cover  Missing
-----
mypackage/__init__.py                1     0   100%
mypackage/calculator.py             14     6    57%    11, 16, 20-23
-----
TOTAL                               15     6    60%

===== 2 passed in 0.03 seconds =====
```

图 37: 结果截图

可以看出漏了 11、16、20-23 行。

4、手动补测试，提高覆盖率。

```

ouc@islouc-vm:~/hw1$ cat > tests/test_calculator.py << 'EOF'
> import pytest
> from mypackage import add, append_item, divide, shout
>
> def test_add():
>     assert add(2, 3) == 5
>     assert add(-1, 1) == 0
>
> def test_divide_normal():
>     assert divide(10, 2) == 5
>
> def test_divide_by_zero():
>     # 覆盖第 11 行：除零抛异常
>     with pytest.raises(ValueError, match="cannot divide by zero"):
>         divide(1, 0)
>
> def test_shout():
>     # 覆盖第 16 行
>     assert shout("hi") == "HI!!!"
>
> def test_append_item_default():
>     # 覆盖 20-23：默认新建列表
>     assert append_item(1) == [1]
>     assert append_item(2) == [2]
>
> def test_append_item_reuse():
>     # 覆盖 20-23：传入已有列表
>     items = ["a"]
>     out = append_item("b", items)
>     assert out == ["a", "b"]
> EOF

```

图 38: 修改后代码

5、重新运行，看覆盖率是否提升。

```

ouc@islouc-vm:~/hw1$ python3 -m pytest tests/ -v --cov=mypackage --cov-report=term
-missing
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0 -- /usr/bin
/python3
cachedir: .pytest_cache
rootdir: /home/ouc/hw1
plugins: cov-5.0.0
collected 6 items

tests/test_calculator.py::test_add PASSED [ 16%]
tests/test_calculator.py::test_divide_normal PASSED [ 33%]
tests/test_calculator.py::test_divide_by_zero PASSED [ 50%]
tests/test_calculator.py::test_shout PASSED [ 66%]
tests/test_calculator.py::test_append_item_default PASSED [ 83%]
tests/test_calculator.py::test_append_item_reuse PASSED [100%]

----- coverage: platform linux, python 3.8.10-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
mypackage/__init__.py                1      0   100%
mypackage/calculator.py             14      0   100%
TOTAL                               15      0   100%

===== 6 passed in 0.03 seconds =====

```

图 39: 结果截图

6、生成 HTML 覆盖率报告

```
ouc@islouc-vm:~/hw1$ python3 -m pytest tests/ --cov=mypackage --cov-report=html
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/hw1
plugins: cov-5.0.0
collected 6 items

tests/test_calculator.py ..... [100%]

----- coverage: platform linux, python 3.8.10-final-0 -----
Coverage HTML written to dir htmlcov

===== 6 passed in 0.03 seconds =====
```

图 40: 覆盖率报告

3.3 实例三：持续集成

- 1、编写 Makefile，封装 CI 检查命令。

```
ouc@islouc-vm:~/hw1$ cat > Makefile << 'EOF'
> .PHONY: check fmt
> check:
> python3 -m ruff format --check . && python3 -m ruff check . && python3 -m pytest
tests/ --cov=mypackage
> fmt:
> Burp Suite Community Edition Format .
> EOF
ouc@islouc-vm:~/hw1$ make check
Makefile:3: *** missing separator. Stop.
ouc@islouc-vm:~/hw1$ sed -i 's/^python3/\\tpython3/' Makefile
```

图 41: Makefile

- 2、正常代码下运行 CI 检查。

```
ouc@islouc-vm:~/hw1$ make check
python3 -m ruff format --check . && python3 -m ruff check . && python3 -m pytest t
ests/ --cov=mypackage
3 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/hw1
plugins: cov-5.0.0
collected 6 items

tests/test_calculator.py ..... [100%]

----- coverage: platform linux, python 3.8.10-final-0 -----
Name                               Stmts  Miss  Cover
-----
mypackage/__init__.py               2      0  100%
mypackage/calculator.py            13      0  100%
-----
TOTAL                               15      0  100%

===== 6 passed in 0.03 seconds =====
```

图 42: 结果截图

- 3、故意引入错误，验证 CI 拦截。在 calculator.py 中添加一个含未使用变量的函数，再次执行 make check。

```

ouc@islouc-vm:~/hw1$ cat >> mypackage/calculator.py << 'EOF'
>
>
> def evil():
>     unused = 999
>     return 1
> EOF
ouc@islouc-vm:~/hw1$
ouc@islouc-vm:~/hw1$ make check
python3 -m ruff format --check . && python3 -m ruff check . && python3 -m pytest tests/ --cov=mypackage
3 files already formatted
F841 Local variable 'unused' is assigned to but never used
--> mypackage/calculator.py:26:5
25 | def evil():
26 |     unused = 999
    |     ^^^^^^
27 |     return 1
    |
help: Remove assignment to unused variable 'unused'

Found 1 error.
No fixes available (1 hidden fix can be enabled with the '--unsafe-fixes' option).
make: *** [Makefile:3: check] Error 1

```

图 43: 结果截图

4、修复错误，恢复 CI 通过，删除 evil 函数，重新执行 make check。

```

ouc@islouc-vm:~/hw1$ cat > mypackage/calculator.py << 'EOF'
> """demo"""
>
> def add(a, b):
>     return a + b
>
> def divide(a, b):
>     if b == 0:
>         raise ValueError("cannot divide by zero")
>     return a / b
>
> def shout(message):
>     return message.upper() + "!!!"
> EOF
ouc@islouc-vm:~/hw1$
ouc@islouc-vm:~/hw1$ make check
python3 -m ruff format --check . && python3 -m ruff check . && python3 -m pytest tests/ --cov=mypackage
3 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/hw1
plugins: cov-5.0.0
collected 6 items

tests/test_calculator.py ..... [100%]

----- coverage: platform linux, python 3.8.10-final-0 -----
Name                Stmts  Miss  Cover
-----
mypackage/__init__.py      2      0   100%
mypackage/calculator.py   13      0   100%
-----
TOTAL                      15      0   100%

===== 6 passed in 0.03 seconds =====

```

图 44: 结果截图

3.4 实例四：grep 正则找危险代码

1、创建 danger.py，包括 4 种 shell=True 变体：

```
ouc@islouc-vm:~/hw1$ cat > danger.py << 'EOF'
> import subprocess
> def good():
>     subprocess.Popen(["ls", "-l"])
> def bad1():
>     subprocess.Popen("ls", shell=True)
> def bad2():
>     subprocess.Popen("rm -rf /tmp/"+name,shell=True,)
> def bad3():
>     subprocess.Popen("ls",shell="True")
> def bad4():
>     subprocess.call("ls",shell = True)
> EOF
```

图 45: danger.py

2、用 grep 正则查找。

```
ouc@islouc-vm:~/hw1$ grep -nE 'subprocess\.Popen.*shell=True' danger.py
5:     subprocess.Popen("ls", shell=True)
7:     subprocess.Popen("rm -rf /tmp/"+name,shell=True,)
```

图 46: 查找结果

3、验证 grep 的漏报。

```
ouc@islouc-vm:~/hw1$ grep -n 'shell="True"' danger.py
9:     subprocess.Popen("ls",shell="True")
ouc@islouc-vm:~/hw1$ grep -n 'subprocess.call' danger.py
11:     subprocess.call("ls",shell = True)
```

图 47: 验证截图

4、用 semgrep 规则查找，编写 semgrep.yml 规则。

```
ouc@islouc-vm:~/hw1$ cat > semgrep.yml << 'EOF'
> rules:
>   - id: dangerous-shell
>     pattern: subprocess.$FUNC(..., shell=True)
>     message: dangerous shell=True
>     languages: [python]
>     severity: ERROR
> EOF
```

图 48: semgrep.yml

执行：

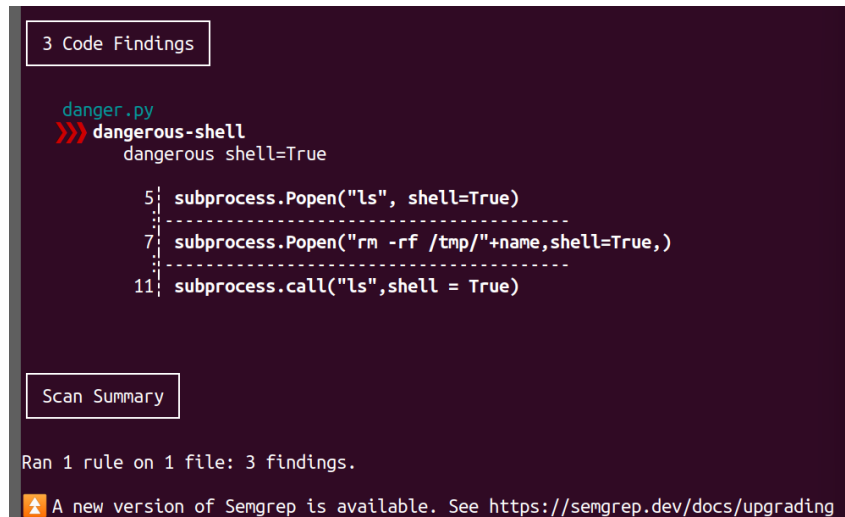


图 49: 结果截图

由此可以看出：semgrep 能抓到 grep 因写法差异漏掉的代码，但抓不到语义变化的 shell="True"。

3.5 实例五：Markdown 项目符号替换

1、编写示例 Markdown 文件。创建 notes.md，包含项目符号、范围、负号、连字符、分隔线。

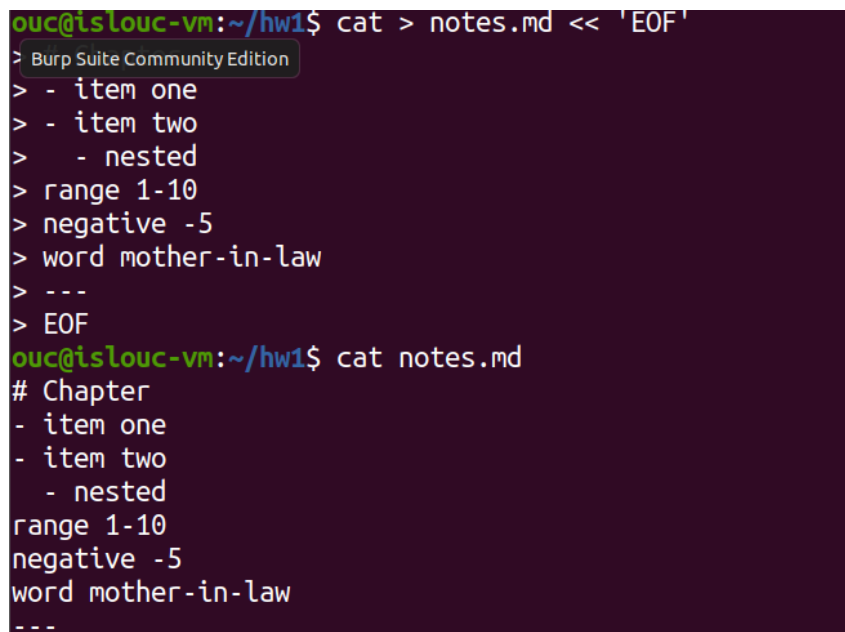


图 50: 文件创建

2、错误做法，全局替换所有“-”。

```
ouc@islouc-vm:~/hw1$ sed 's/-/*/' notes.md
# Chapter
* item one
* item two
  * nested
range 1*10
negative *5
word mother*in*law
***
```

图 51: 结果截图

所有-都被替换，范围 1-10、负号-5、连字符 mother-in-law、分隔线—全被破坏。

3、正确做法，只替换行首项目符号。

```
ouc@islouc-vm:~/hw1$ sed 's/^\([[:space:]]*\)- /\1* /' notes.md
# Chapter
* item one
* item two
  * nested
range 1-10
negative -5
word mother-in-law
---
```

图 52: 结果截图

只替换了行首项目符号，其他连字符用途全部保留。

3.6 实例六：正则提取 JSON 的 name

1、编写正则演示脚本，创建 regex_demo.py，测试 5 种正则：

```
ouc@islouc-vm:~/hw1$ cat > regex_demo.py << 'EOF'
> import json,re
> s='{"name":"Alyssa P. Hacker","college":"MIT"}'
> print("朴素 .*:",re.search(r'"name":.*"',s).group(1))
> print("否定类:",re.search(r'"name":("[^"]*)"',s).group(1))
> e=r'{"name":"Alyssa \"Alice\" Hacker","college":"MIT"}'
> print("转义引号旧:",re.search(r'"name":("[^"]*)"',e).group(1))
> print("转义引号新:",re.search(r'"name":((?:\\.|"[^"]\\")*)"',e).group(1))
> print("JSON解析:",json.loads(s)["name"])
> EOF
```

图 53: regex_demo.py

运行并观察结果。

```
ouc@islouc-vm:~/hw1$ python3 regex_demo.py
朴素 .*: "name":"Alyssa P. Hacker","college"
否定类: Alyssa P. Hacker
转义引号旧: Alyssa \
转义引号新: Alyssa \"Alice\" Hacker
JSON解析: Alyssa P. Hacker
```

图 54: 结果截图

2、命令行 JSON 解析器。

```
ouc@islouc-vm:~/hw1$ echo '{"name":"Grace Hopper","college":"Yale"}' | python3 -c
"import json,sys;print(json.load(sys.stdin)['name'])"
Grace Hopper
```

图 55: 结果截图

结果: Grace Hopper

3.7 实例七: Makefile clean 目标

1、创建 paper.tex。

```
ouc@islouc-vm:~/hw1$ cat > paper.tex << 'EOF'
> \documentclass{article}
> \begin{document}
> Hello
> \end{document}
> EOF
```

图 56: paper.tex

2、编写 Makefile, 定义 paper.pdf 构建目标和 clean 清理目标。

```
ouc@islouc-vm:~/hw1$ cat > Makefile << 'EOF'
> .PHONY: all check fmt clean
> all: paper.pdf
> paper.pdf: paper.tex
> pdflatex paper.tex
> check:
> python3 -m ruff format --check . && python3 -m ruff check . && python3 -m pytest
tests/ --cov=mypackage
> fmt:
> python3 -m ruff format .
> clean:
> rm -f paper.pdf paper.aux paper.log
> EOF
ouc@islouc-vm:~/hw1$ sed -i 's/^pdflatex/\tpdflatex;/s/^python3/\tpython3;/s/^rm/\t
rm/' Makefile
```

图 57: Makefile

3、模拟构建产物。

```
ouc@islouc-vm:~/hw1$ touch paper.pdf paper.aux paper.log
ouc@islouc-vm:~/hw1$ ls paper.*
paper.aux  paper.log  paper.pdf  paper.tex
```

图 58: 结果截图

当前目录有 paper.aux、paper.log、paper.pdf、paper.tex 四个文件。

4、运行 clean。

```
ouc@islouc-vm:~/hw1$ make clean
rm -f paper.pdf paper.aux paper.log
```

图 59: 结果截图

5、验证清理结果

```
ouc@islouc-vm:~/hw1$ ls paper.*
paper.tex
```

图 60: 清理结果

只剩 paper.tex，构建产物已全部删除。

3.8 实例八：Rust 版本要求语法

1、完整的 Cargo.toml 展示。

```
ouc@islouc-vm:~/hw1$ cat rust_demo/Cargo.toml
[package]
name = "demo"
version = "0.1.0"
edition = "2021"
[dependencies]
rand = "^0.8.5"
tokio = "~1.25.0"
log = "0.4.*"
openssl = "=0.10.57"
bytes = ">=1.0.1"
thiserror = ">=1.0, <2.0"
regex = "1.0.0 - 1.9.0"
lazy_static = "*"
```

图 61: Cargo.toml

表 1: Rust 版本要求语法

语法	示例	含义	适用场景
^	^0.8.5	>=0.8.5, <0.9.0	默认，允许兼容升级
~	~1.25.0	>=1.25.0, <1.26.0	只吃补丁，最严
*	0.4.*	>=0.4.0, <0.5.0	主版本对就行
=	=0.10.57	精确版本	只测试过这个版本
>=	>=1.0.1	大于等于	避开有 bug 的版本
多个要求	>=1.0, <2.0	区间	需要新功能但怕破坏性变更
连字符	1.0.0 - 1.9.0	闭区间	明确验证过的范围

3.9 实例九：pre-commit 钩子跑 make paper.pdf

1、编写 pre-commit 钩子，创建.git/hooks/pre-commit。

```
ouc@islouc-vm:~/hw1$ cat > .git/hooks/pre-commit << 'EOF'
> #!/bin/bash
> echo "[pre-commit] running make paper.pdf"
> if ! make paper.pdf; then
>     echo "[pre-commit] build failed, commit rejected"
>     exit 1
> fi
> echo "[pre-commit] build passed"
> exit 0
> EOF
```

图 62: pre-commit

赋予执行权限,chmod +x .git/hooks/pre-commit。

2、正常构建,提交通过,编写正常的 paper.tex,执行 git commit。

```
ouc@islouc-vm:~/hw1$ cat > paper.tex << 'EOF'
> \documentclass{article}
> \begin{document}
> Hello
> \end{document}
> EOF
ouc@islouc-vm:~/hw1$ git add -A
ouc@islouc-vm:~/hw1$ git commit -m "test build hook"
[pre-commit] running make paper.pdf
pdflatex paper.tex
This is pdfTeX, Version 3.14159265-2.6-1.40.20 (TeX Live 2019/Debian) (preloaded f
ormat=pdflatex)
restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2020-02-02> patch level 2
L3 programming layer <2020-02-14>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2019/12/20 v1.4l Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfmode.def)
(./paper.aux) [1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}]
(./paper.aux) </usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
r10.pfb>
Output written on paper.pdf (1 page, 10456 bytes).
Transcript written on paper.log.
[pre-commit] build passed
[master 30194d3] test build hook
2 files changed, 16 insertions(+), 15 deletions(-)
```

图 63: 结果截图

3、添加普通文字,验证不影响编译

```
ouc@islouc-vm:~/hw1$ echo "BROKEN" >> paper.tex
ouc@islouc-vm:~/hw1$ git add -A
ouc@islouc-vm:~/hw1$ git commit -m "test broken"
[pre-commit] running make paper.pdf
pdflatex paper.tex
This is pdfTeX, Version 3.14159265-2.6-1.40.20 (TeX Live 2019/Debian) (preloaded f
ormat=pdflatex)
restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2020-02-02> patch level 2
L3 programming layer <2020-02-14>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2019/12/20 v1.4l Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfmode.def)
(./paper.aux) [1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}]
(./paper.aux) </usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
r10.pfb>
Output written on paper.pdf (1 page, 10456 bytes).
Transcript written on paper.log.
[pre-commit] build passed
[master ace5c0c] test broken
3 files changed, 2 insertions(+), 1 deletion(-)
```

图 64: 结果截图

LaTeX 不报错，构建成功，提交通过。说明普通文字改动不影响编译。

4、故意破坏构建，验证钩子拦截，在 paper.tex 中加入未定义命令，执行 git commit。

```
ouc@islouc-vm:~/hw1$ cat > paper.tex << 'EOF'
> \documentclass{article}
> \begin{document}
> \undefinedcommand
> \end{document}
> EOF
ouc@islouc-vm:~/hw1$ git add -A
ouc@islouc-vm:~/hw1$ git commit -m "test broken build"
[pre-commit] running make paper.pdf
pdflatex paper.tex
This is pdfTeX, Version 3.14159265-2.6-1.40.20 (TeX Live 2019/Debian) (preloaded f
ormat=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2020-02-02> patch level 2
L3 programming layer <2020-02-14>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2019/12/20 v1.4l Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfmode.def)
(./paper.aux)
! Undefined control sequence.
l.3 \undefinedcommand
?
! Emergency stop.
l.3 \undefinedcommand

! ==> Fatal error occurred, no output PDF file produced!
Transcript written on paper.log.
make: *** [Makefile:4: paper.pdf] Error 1
[pre-commit] build failed, commit rejected
```

图 65: 拒绝截图

pdflatex 报错! Undefined control sequence., make 退出码非 0, 钩子输出 build failed, commit rejected, 提交被拒绝。

5、修复构建，提交成功，恢复正常的 paper.tex，执行 git commit。

```
ouc@islouc-vm:~/hw1$ cat > paper.tex << 'EOF'
> \documentclass{article}
> \begin{document}
> Hello
> \end{document}
> EOF
ouc@islouc-vm:~/hw1$ git add -A
ouc@islouc-vm:~/hw1$ git commit -m "fix paper"
[pre-commit] running make paper.pdf
pdflatex paper.tex
This is pdfTeX, Version 3.14159265-2.6-1.40.20 (TeX Live 2019/Debian) (preloaded f
ormat=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2020-02-02> patch level 2
L3 programming layer <2020-02-14>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2019/12/20 v1.4l Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfmode.def)
(./paper.aux) [1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}]
(./paper.aux) </usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
r10.pfb>
Output written on paper.pdf (1 page, 10456 bytes).
Transcript written on paper.log.
[pre-commit] build passed
[master cd228e2] fix paper
3 files changed, 14 insertions(+), 16 deletions(-)
```

图 66: 结果截图

pdflatex 成功构建，输出 build passed, 提交成功。

3.10 实例十：shellcheck+GitHub Action

1、安装 shellcheck

```
ouc@islouc-vm:~/hw1$ sudo apt install -y shellcheck
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  shellcheck
0 upgraded, 1 newly installed, 0 to remove and 64 not upgraded.
```

图 67: 结果截图

2、编写一个有问题的 shell 脚本，让 shellcheck 抓错：

```
ouc@islouc-vm:~/hw1$ cat > test.sh << 'EOF'
> #!/bin/bash
> echo $1
> ls $2
> if [ $1 = "hello" ]
> then
>     echo "world"
> fi
> EOF
ouc@islouc-vm:~/hw1$ chmod +x test.sh
ouc@islouc-vm:~/hw1$ shellcheck test.sh

In test.sh line 2:
echo $1
    ^-- SC2086: Double quote to prevent globbing and word splitting.

Did you mean:
echo "$1"

In test.sh line 3:
ls $2
    ^-- SC2086: Double quote to prevent globbing and word splitting.

Did you mean:
ls "$2"

In test.sh line 4:
if [ $1 = "hello" ]
    ^-- SC2086: Double quote to prevent globbing and word splitting.

Did you mean:
if [ "$1" = "hello" ]

For more information:
  https://www.shellcheck.net/wiki/SC2086 -- Double quote to prevent globbing ...
```

图 68: 结果截图

报 3 个 SC2086（变量未加引号，可能导致 globbing 和分词）。

4、修复脚本，重新检查。

```
ouc@islouc-vm:~/hw1$ cat > test.sh << 'EOF'
> #!/bin/bash
> echo "$1"
> ls "$2"
> if [ "$1" = "hello" ]
> then
>     echo "world"
> fi
> EOF
ouc@islouc-vm:~/hw1$ shellcheck test.sh
```

图 69: 结果截图

无输出，说明检查通过。

5、编写 GitHub Action 配置。

```
ouc@islouc-vm:~/hw1$ mkdir -p .github/workflows
ouc@islouc-vm:~/hw1$ cat > .github/workflows/shellcheck.yml << 'EOF'
> name: ShellCheck
> on: [push, pull_request]
> jobs:
>   shellcheck:
>     runs-on: ubuntu-latest
>     steps:
>       - uses: actions/checkout@v4
>       - name: Run ShellCheck
>         run: |
>           find . -name "*.sh" -exec shellcheck {} +
> EOF
```

图 70: 结果截图

把代码 push 到 GitHub 后，自动运行 shellcheck 检查所有.sh 文件。

4 解题感悟

通过此次实验，我进一步理解了代码代码质量、元编程和大杂烩。学习到 formatter 与 linter 将代码风格和低级错误自动化，pre-commit 钩子把检查前置到提交前，CI 让每次推送都自动验证，覆盖率工具精准指出未测试代码行，semgrep 基于 AST 匹配比裸 grep 更可靠，正则表达式在处理转义引号时的脆弱性让我明白复杂解析应交给专用解析器，而 pre-commit 钩子和 Makefile clean 目标则体现了“自动化重复劳动、把精力留给思考”的工程思想。

5 commit 记录

Yan-cheng-yi Add files via upload 5d1a48b · 1 minute ago 17 Commits		
paper	Add files via upload	3 weeks ago
第一次课后实验结果图	Add files via upload	3 weeks ago
第三周实验课外截图	Add files via upload	4 days ago
第二次课外实验截图	Add files via upload	last week
第四周课外实验截图	Add files via upload	7 minutes ago
第四次课内实验截图	Add files via upload	last week
课内images2	Add files via upload	2 weeks ago
README.md	Initial commit	3 weeks ago
第一次课后作业命令记录.txt	Add files via upload	3 weeks ago
第三周实验.pdf	Add files via upload	4 days ago
第三周课外命令.txt	Add files via upload	1 minute ago
第三次课内实验命令.txt	Add files via upload	2 weeks ago
第二次实验报告.pdf	Add files via upload	last week
系统开发工具基础第一周第二周课内实验命令...	Add files via upload	3 weeks ago
系统开发工具基础第一周课程实验.docx	Add files via upload	3 weeks ago
系统开发工具基础第一次作业.pdf	Add files via upload	2 weeks ago
系统开发模板.zip	Add files via upload	3 weeks ago

图 71: commit 截图